04. 알고리즘과 조건·반복문

제조 바이브코딩 프로토타입 캠프



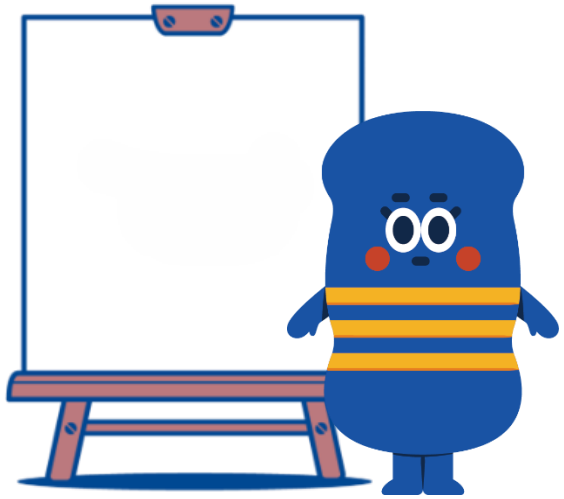

시작하기

학습 목표





시작하기



1. 알고리즘이 '문제를 푸는 순서'임을 이해하고, 동작을 순서도로 그릴 수 있다.

2. 조건문과 반복문의 개념을 알고, AI가 만든 코드에서 구조를 찾아 읽을 수 있다.


알고리즘이란?

알고리즘이란?





칫솔과 치약을
준비한다



치약 뚜껑을
연다



치약을 짠다



구석구석
양치를 한다



입을 헹군다



알고리즘(Algorithm)은 주어진 문제를 논리적으로 해결하기 위해 필요한 절차, 방법, 명령어들을 모아놓은 것을 말합니다.

알고리즘 순서도 (1)



순서도 도형	도형 명칭	설명	사용 예시
	시작/끝 (terminator)	순서도의 시작과 끝을 나타냄	프로그램 시작, 종료
	처리(Process)	작업이나 연산 등 처리 과정을 나타냄	계산 수행, 데이터 저장, 변수 대입 등
	입력/출력 (Input/Output)	데이터의 입력 또는 출력(입출력)을 나타냄	데이터 입력, 결과 출력 등
	판단(Decision)	조건 판단이나 분기(예/아니오)를 나타냄	조건 검사, 참/거짓 분기 등
	흐름선(Flow Line)	작업 흐름의 방향과 순서를 나타냄	다음 단계로 이동, 흐름 연결
	연결자(Connector)	순서도의 다른 위치로 흐름을 연결할 때 사용 (같은 페이지 내 연결)	다른 위치의 흐름 연결
	페이지 연결자 (Off-page Connector)	다른 페이지에 있는 순서도와 연결할 때 사용	다른 페이지로 흐름 연결
	문서(Document)	문서나 보고서 동의 문서를 나타냄	보고서 출력, 문서 생성 등
	미리 정의된 과정 (Predefined Process)	다른 순서도(서브 루틴)나 정의된 과정을 나타냄	서브 루틴 호출, 모듈 실행 등

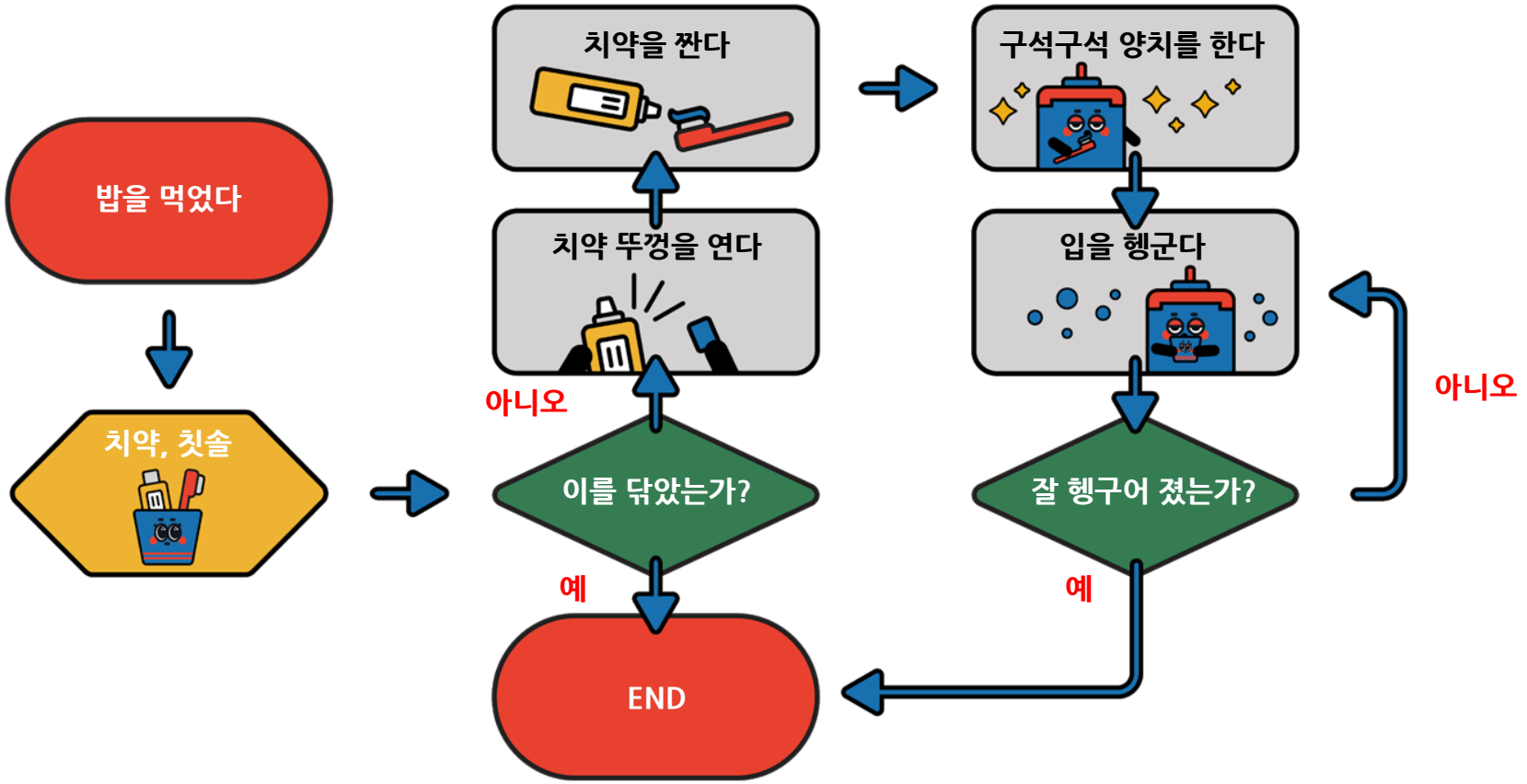


순서도는 도형을 연결하여 프로그램의 흐름을 표현하는 방법입니다.

알고리즘 순서도 (2)

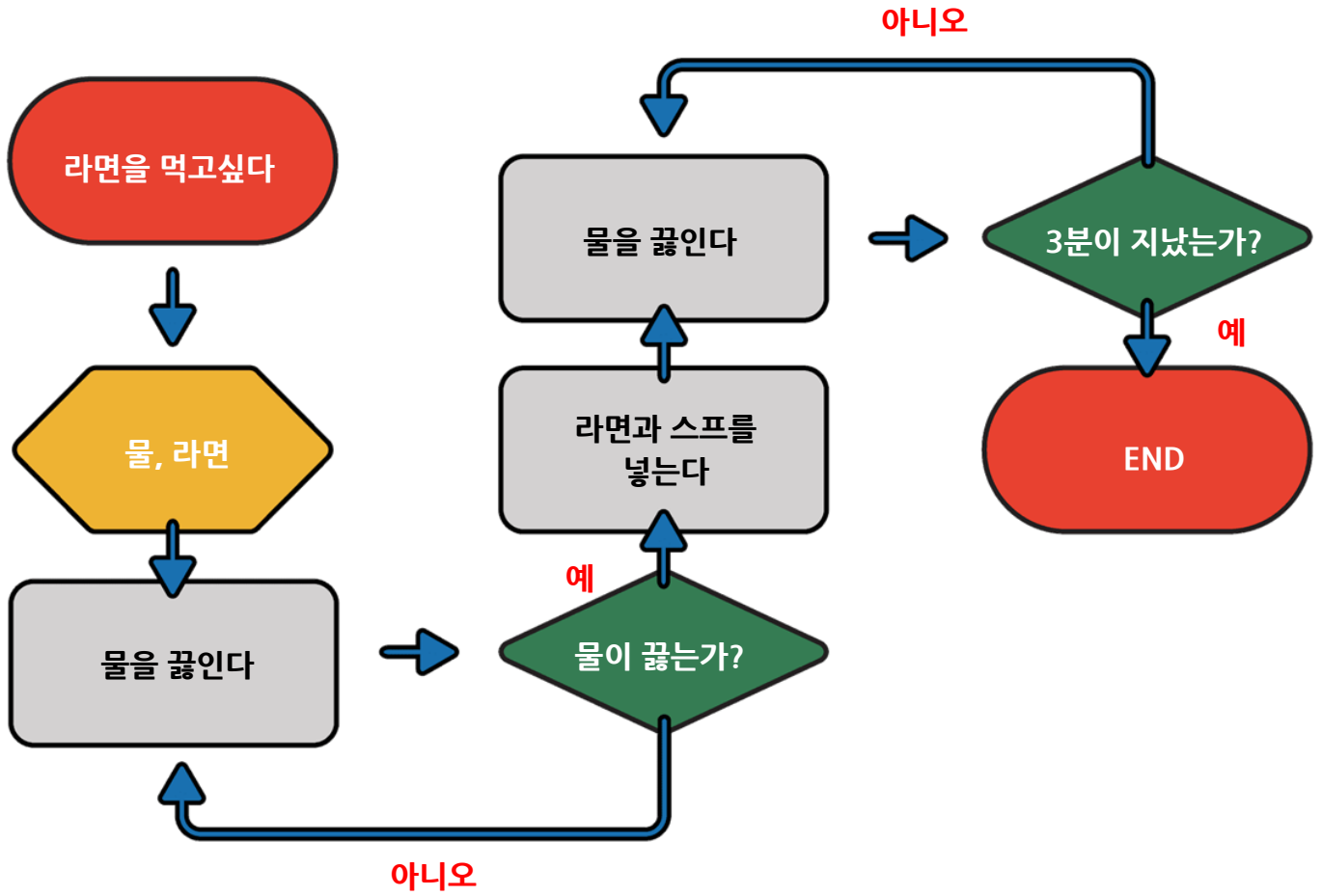





밥을 먹었다
치약, 칫솔
이를 닦았는가?
아니오
예
치약을 짠다
치약 뚜껑을 연다
END
구석구석 양치를 한다
입을 헹군다
잘 헹구어 졌는가?
아니오
예

알고리즘 순서도 (3)







알고리즘이란?

순차구조란?



앞으로 1보 가기

왼쪽으로 2보 가기

앞으로 1보 가기

오른쪽으로 3보 가기

순차구조

'순'서대로

'차'례차례



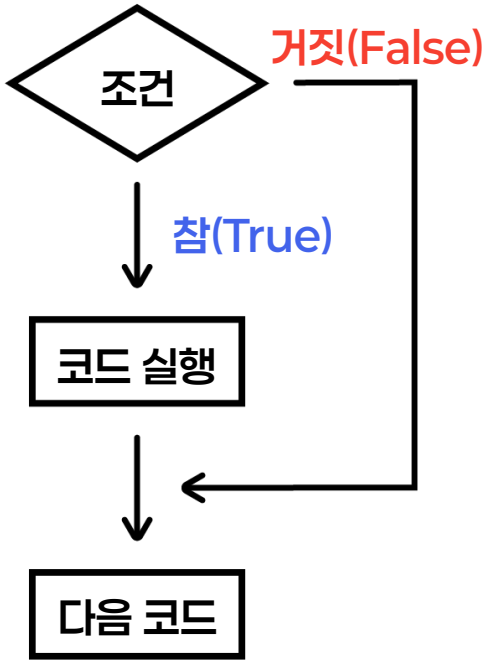
알고리즘이란?

한 방향으로 쭉 뻗은 직선처럼,
정해진 순서에 따라 차례대로 실행하는 알고리즘을 **순차구조**라고 합니다.

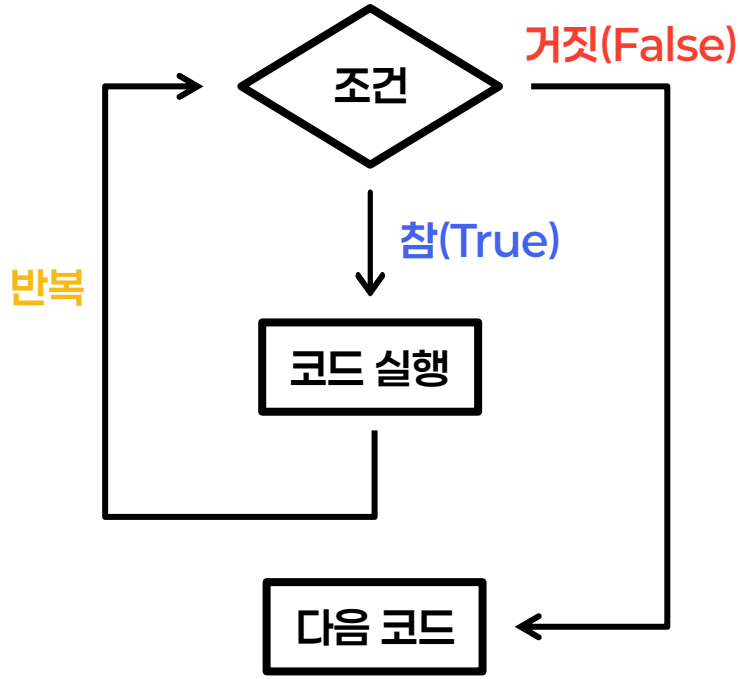
제어문이란?



조건문 순서도



반복문 순서도



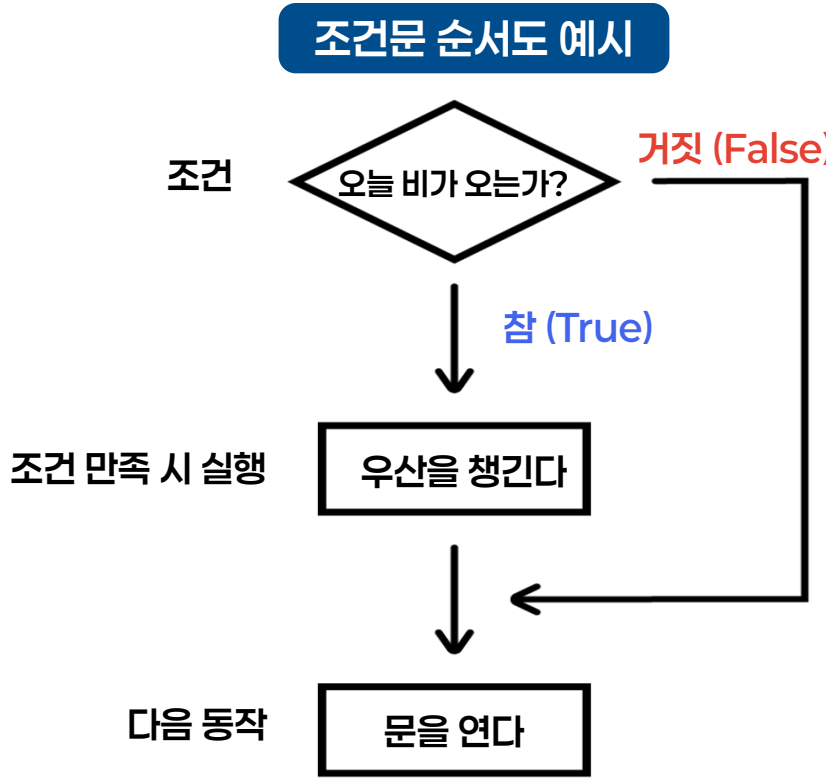


알고리즘이란?

제어문은 프로그램의 흐름을 제어할 때 사용됩니다.

조건문이란?





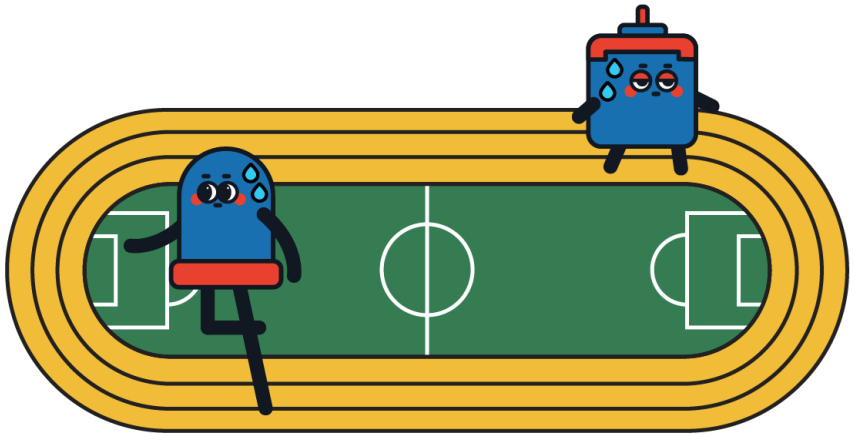
조건문은 특정 조건에 따라 참(True)인지 거짓(False)인지 판단하고,
조건이 참인 경우에만 조건문 안의 코드를 실행합니다.



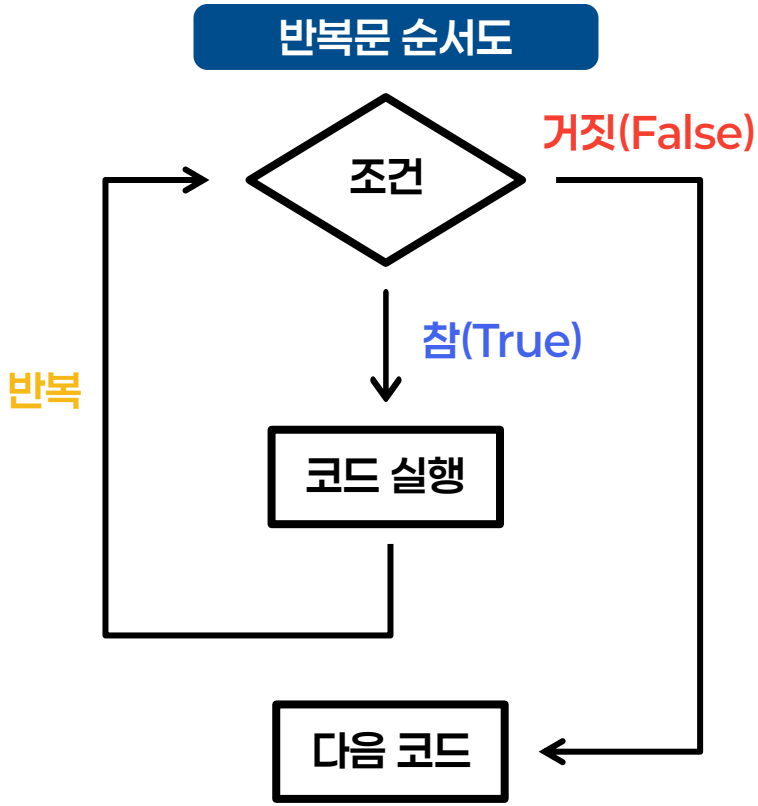
알고리즘이란?

반복문이란?





x N



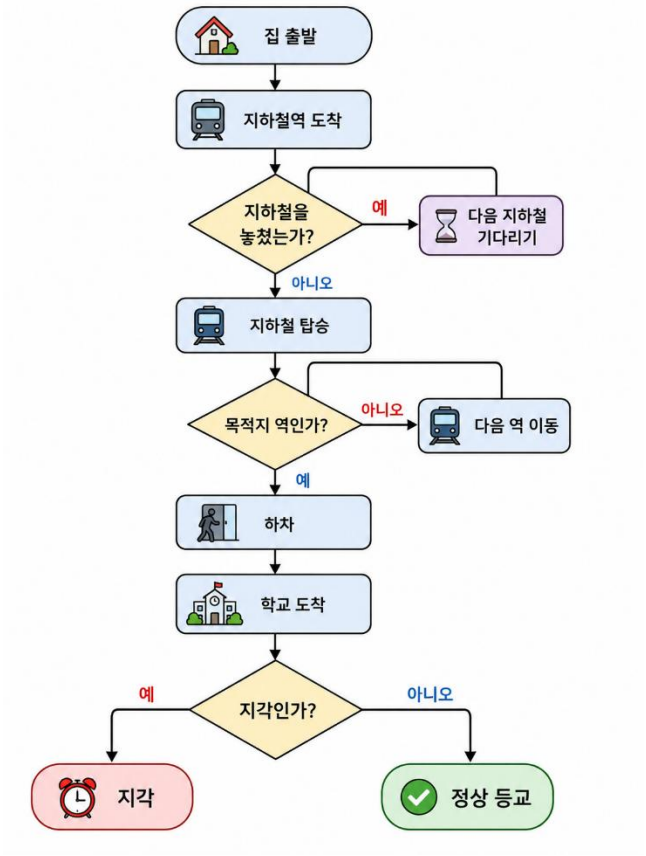


알고리즘이란?

반복문은 조건을 만족하는 동안 반복문 안의 코드를 반복해서 실행하고,
조건을 만족하지 않으면 반복문에서 벗어나 다음 코드를 실행합니다.

순서도 그려보기





알고리즘이란?

상황을 설정하여 알고리즘을 순서도로 표현해봅니다.


조건문

조건문의 종류



조건문의 종류

① if

② if ~ else

③ if ~ else if ~ else

④ switch ~ case



조건문의 종류에 따라 프로그램의 흐름을 다르게 제어할 수 있습니다.

if 조건문



```
1   if (조건1) {
2
3
4       명령;
5
6
7   }
8
9
10
11
12  다음 코드;
13
14
15  다음 코드;
16
17
18  …
19
20
21
```

조건1이 참이면
if문 괄호{ } 안의 명령을 수행하고,
다음 코드로 넘어갑니다.

조건을 만족하지 않으면
명령을 수행하지 않고
조건문 밖의 다음 코드로 넘어갑니다.



if ~ else 조건문



```
1   if (조건1) {
2
3
4       명령1;
5
6
7   }
8
9
10  else {
11
12
13      명령2;
14
15  }
16
17
18  다음 코드;
19
20
21  다음 코드;
```

조건1을 만족한다면
if문 { } 안의 명령1을 수행하고
다음 코드로 넘어갑니다.

조건1을 만족하지 않으면
else문 { } 안에 있는 명령2를 수행하고
다음 코드로 넘어갑니다.

비교 조건이 2가지 일 때 사용하면 편리합니다.



if ~ else if ~ else 조건문



```
1   if (조건1) {
2       명령1;
3   }
4
5   else if (조건2) {
6       명령2;
7   }
8
9   else if (조건3) {
10      명령3;
11  }
12
13  else {
14      명령4;
15  }
16
17  다음 코드;
18
19  다음 코드;
20
21
```

if문의 조건1부터 else if문의 조건3까지
각각의 조건이 참인지 거짓인지 판별 후
참이면 명령 수행 후 다음 코드로 넘어가고,
거짓이면 다음 조건으로 이동합니다.

앞선 조건이 모두 만족하지 않으면
else 문의 코드를 실행합니다.



조건문

산술 연산자



연산자	의미	사용 예시
+	덧셈	a + b
-	뺄셈	a – b
*	곱셈	a * b
/	나눗셈	a / b
%	나머지	a % b

조건을 설정할 때 연산자를 사용할 수 있습니다.



증감 연산자



연산자	다른 표현	의미
a++	a = a + 1;	명령 수행 후, a의 값 1 증가
	a += 1;	
a--	a = a – 1;	명령 수행 후, a의 값 1 감소
	a –= 1;	

a++ 는 a의 값에 1을 더합니다. a-- 는 a의 값에 1을 뺍니다.

예시) a의 값이 5일 때 a++; 을 하면 a의 값 5에 1을 더해, a는 6이 됩니다.



조건문

관계 연산자



연산자	사용 예시	의미
>	a > b	a가 b보다 크다
<	a < b	a가 b보다 작다
>=	a >= b	a가 b보다 크거나 같다
<=	a <= b	a가 b보다 작거나 같다
!=	a != b	a와 b가 같지 않다
==	a == b	a와 b가 같다

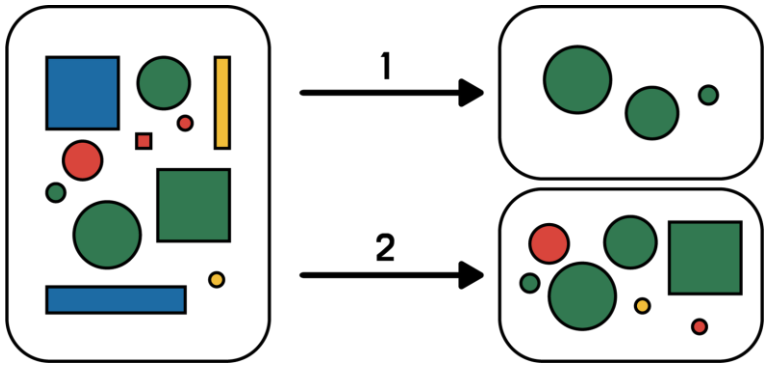


관계 연산자는 좌우의 값을 비교하여 참(True = 1)과 거짓(False = 0)을 결정하는 연산자입니다.

논리 연산자



연산자	사용 예시	의미
&&	a > b && c > d	a > b 이고 c > d이다. (AND)
||	a > b || c > d	a > b 또는 c > d이다. (OR)
!	!a	a가 아니다 (NOT)



Circle and green

Circle or green

논리 연산자는 좌우 조건식이나 값을 논리적으로 판단하여 참과 거짓을 결정합니다.



조건문 사용 예시



led_switch_count.ino

```
1   int count = 0;
2   int LED = 5;
3   int SW = 2;
4
5   void setup(){
6       pinMode(LED, OUTPUT);
7       pinMode(SW, INPUT_PULLUP);
8   }
9
10  void loop(){
11      if(digitalRead(SW) == LOW){
12          delay(200);
13          count++;
14      }
15      if(count >= 3){
16          digitalWrite(5,HIGH);
17      }
18      else {
19          digitalWrite(5,LOW);
20      }
21  }
```

Line 11~14

스위치가 눌릴 때마다
count 값이 1씩 증가합니다.

Line 15~20

스위치가 3번 이상 눌렸으면 LED가 켜지고,
아니면 LED가 꺼집니다.



조건문

조건문 활용하기 (1)





모티는 직코 마을의 가로등을 관리하고 있습니다.
그런데 어두워질 때마다 가로등을 켜는 작업이 번거롭다고 하네요.
모티와 함께 밝을 때에는 조명을 껐다가 어두워지면 자동으로 불을 하나씩 켜는 스마트 가로등을 만들어 봅시다.

조도센서값이 300 미만일 때는 네오픽셀을 끄고,

300 이상 700 이하 일때는 네오픽셀 1,2번째를 흰색으로 켜고,

700 초과일때는 네오픽셀을 모두 초록색으로 켜보아요.



조건문

조건문을 활용해 스마트 가로등을 만들어 봅니다.

조건문 활용하기 (1)



Gemini 프롬프트 예시

지침:
너는 아두이노 하드웨어 펌웨어 개발자야. 밝을 때에는 조명이 꺼지고, 어두워지면 자동으로 불을 켜는 스마트 가로등을 만들어보려고 해. 아래의 제약조건과 출력문 형태에 맞추어 아두이노 IDE 기준으로 코드를 작성해줘.

제약조건:
- **하드웨어:** 조도센서 A0번 핀, 네오픽셀(LED 4개) 7번 핀
- **코드 요청:** 조도센서 값이 300 미만일 때는 네오픽셀을 끄고, 조도센서 값이 300 이상 700 이하일 때는 네오픽셀 1, 2번째 LED를 흰색으로 켜고, 조도센서 값이 700 초과일 때는 네오픽셀 LED를 모두 초록색으로 켜는 코드
- **라이브러리:** 네오픽셀 Adafruit Neopixel

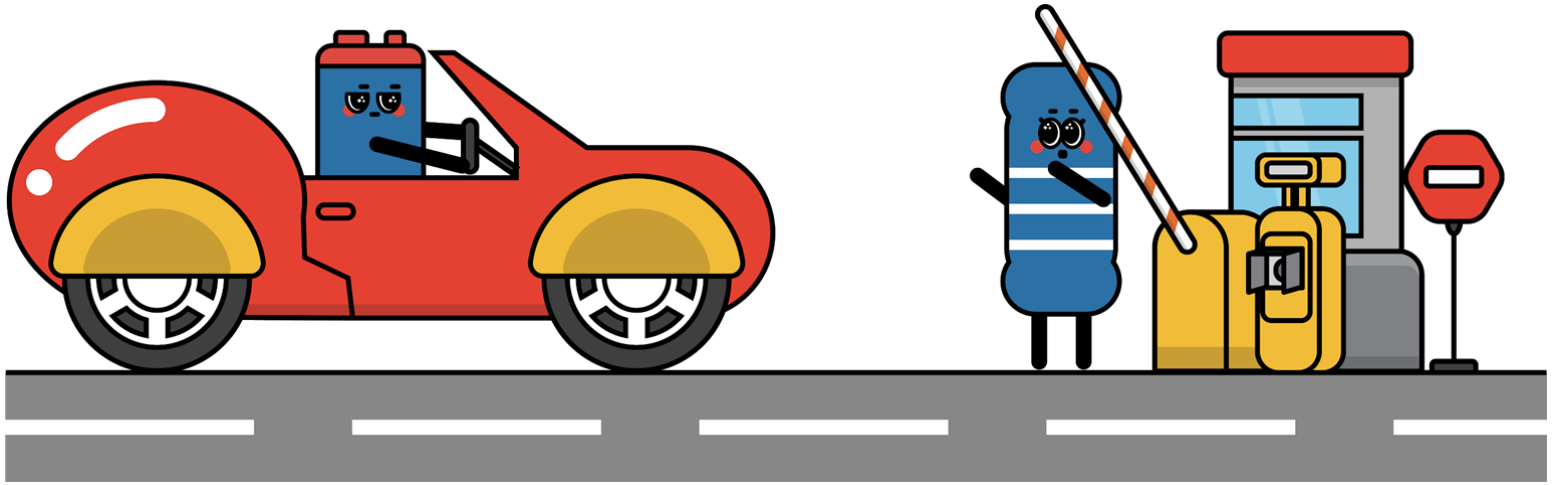
출력문:
- 코드 및 코드에 대한 설명
- 필요 시, 코드 구현을 위해 사용자에게 추가 질의

스마트 가로등 코드 구현을 위한 프롬프트를 작성해봅니다.



조건문 활용하기 (2)





초음파센서를 활용하여 거리가 20cm 이하이면 서보모터를 90도 회전시켜 차량을 내보내고,
2초 뒤 다시 측정하여 20cm 초과면 서보모터가 0도가 되면서 차단기를 닫아주세요.
문이 열리고 닫힐 때마다 open과 close라는 글씨를 LCD에 출력해주세요.

조건문을 활용해 차량 차단기를 만들어 봅니다.



조건문 활용하기 (2)



Gemini 프롬프트 예시

지침:
너는 아두이노 하드웨어 펌웨어 개발자야. 아두이노로 차량차단기를 만들어보려고 해. 아래의 제약조건과 출력문 형태에 맞추어 아두이노 IDE 기준으로 코드를 작성해줘.

제약조건:
- **하드웨어:** 초음파센서 Trig 13번 핀, Echo 12번 핀, 서보모터 8번 핀, LCD SDA A4번 핀, SCL A5번 핀
- **코드 요청:** 초음파센서로 측정한 거리가 20cm 이하이면 서보모터를 90도로 회전시키고, 20cm 초과면 서보모터를 0도로 회전. 서보모터 각도가 90도가 될 때는 LCD에 'OPEN' 이라는 글씨를 출력하고, 0도가 될 때는 LCD에 'CLOSE'라는 글씨를 출력.
- **라이브러리:** 서보모터 Servo, LCD LiquidCrystal_I2C

출력문:
- 코드 및 코드에 대한 설명 (필요 시, 코드 구현을 위해 사용자에 추가 질의)

차량 차단기 코드 구현을 위한 프롬프트를 작성해봅니다.




반복문

반복문의 종류



반복문의 종류

① while

② for



반복문

반복문의 종류에 따라 프로그램의 흐름을 다르게 제어할 수 있습니다.

while 반복문



```
1  while (조건) {
2
3      명령;
4
5  }
6
```

조건을 만족하는 동안
명령을 계속 반복합니다.

while 구문은 **조건식이 참인 동안만 문장을 반복**해서 실행합니다.

조건을 만족하는 경우에는 while 구문 안의 명령을 실행하고
조건을 만족하는지 다시 판단합니다.

조건식의 값이 거짓인 경우에는 while 구문 안의
명령을 실행하지 않고 구문 밖으로 빠져나옵니다.



while 반복문 사용 예시



led_blink_while.ino

```
1  void setup() {
2
3      pinMode(5, OUTPUT);
4
5      int a = 0;
6
7      while(a < 5){
8          digitalWrite(5,HIGH);
9          delay(1000);
10         digitalWrite(5, LOW);
11         delay(1000);
12
13         a++;
14     }
15 }
16
17 void loop() {
18 }
19
20
21
```

Line 7~14

a < 5 조건을 만족하는 동안
while 구문 안의 명령을 반복하면서
a 의 값에 1씩 더하게 됩니다.

a 가 5 이상이 되면,
반복문을 실행하지 않기 때문에

LED를 깜빡이는 동작은 총 5번
반복하게 됩니다.



for 반복문



```
for (초기 값; 조건; 증가/감소 값) {

  명령;

}
```

for 구문을 반복할 때마다
초기 값에 증가 값만큼 더해줍니다.

초기값이 조건을 만족하는 동안
명령을 계속 반복합니다.



for 구문은 초기값이 조건을 만족할 때까지 반복합니다.

for 반복문 사용 예시



led_blink_for.ino

```
1   void setup() {
2
3       pinMode(5, OUTPUT);
4
5       for (int a = 0; a < 5; a++) {
6           digitalWrite(5,HIGH);
7           delay(1000);
8           digitalWrite(5, LOW);
9           delay(1000);
10      }
11  }
12
13  void loop() {
14  }
15
16
17
18
19
20
21
```

Line 5~10

a 의 초기값은 0입니다.
for 구문 안의 코드를 한 번 수행할 때마다
a++ 연산으로 a의 값이 1씩 증가합니다.

a = 4일 때 까지 실행되므로
LED는 총 5번 깜빡이게 됩니다.



반복문


정리하기

코드 해석하기 (1)



```
if (temperature >= 30.0 && humidity >= 70.0) {
  digitalWrite(redLED, HIGH);
  digitalWrite(yellowLED, LOW);
  tone(buzzerPin, 1000);
}

else if (temperature >= 28.0 || humidity >= 60.0) {
  digitalWrite(redLED, LOW);
  digitalWrite(yellowLED, HIGH);
  noTone(buzzerPin);
}

else {
  digitalWrite(redLED, LOW);
  digitalWrite(yellowLED, LOW);
  noTone(buzzerPin);
}
```

다음 코드의 조건을 보고 어떤 코드인지 해석해봅니다.



정리하기

코드 해석하기 (2)



```
for (int bright = 0; bright <= 255; bright += 5) {

  analogWrite(ledPin, bright);
  delay(30);


}
for (int bright = 255; bright >= 0; bright -= 5) {

  analogWrite(ledPin, bright);
  delay(30);


}
```

다음 코드의 조건을 보고 어떤 코드인지 해석해봅니다.



정리하기

다음 시간에 만나요